

SmartCity

Sistema Distribuido com Sockets e Protocol Buffers

Distribuicao de Processos e Dados — UFC

Prof. Dr. Paulo A. L. Rego

Carlos Henriky Vieira Sousa **514128**

Jose Enilson Mesquita da Silva **497562**

Maria Eduarda de Sousa Pereira **510976**

06 de Junho de 2026

Arquitetura do Sistema

Tres processos independentes em tres linguagens diferentes, comunicando via **TCP**, **UDP unicast** e **UDP multicast**, com todas as mensagens serializadas em **Protocol Buffers**.

Componente	Linguagem / Build	Responsabilidade
Gateway Inteligente	Python 3.14	Hub central: coleta, persiste, controla e expoe API
Fontes de Dados	C (GCC 15 + protobuf-c 1.5)	5 processos: 2 sensores + 3 atuadores urbanos
Cliente Analitico	Java 21 + Maven 3.9	Menu interativo: consultas, comandos e analytics

Canais de comunicacao (5 portas)

Protocolo	Porta	Direcao	Mensagens
UDP Multicast	224.1.1.1:5007	Gateway → Fontes	DiscoveryRequest (broadcast de descoberta)
UDP Unicast	0.0.0.0:6002	Fontes → Gateway	DiscoveryResponse (cont.) + Leitura (dados)
TCP	0.0.0.0:6000	Fontes ↔ Gateway	DiscoveryResponse (ctrl.) + Comando + RespostaComando
TCP	0.0.0.0:6001	Cliente ↔ Gateway	RequisicaoCliente + RespostaGateway
HTTP REST	:6003	Browser ↔ Gateway	JSON — Dashboard Web com Chart.js

Protocol Buffers — Definicoes das Mensagens

Arquivo unico SmartCity/proto/cidade.proto define **7 mensagens**. Compilado para Python (cidade_pb2.py), C (cidade.pb-c.c/.h) e Java (Cidade.java) com **protoc 4.34.1**.

DiscoveryRequest (Gateway→Fontes, multicast)

```
message DiscoveryRequest {
  string gateway_ip = 1;
  int32 gateway_port = 2; // TCP
}
```

DiscoveryResponse (Fonte→Gateway)

```
message DiscoveryResponse {
  string source_id = 1;
  string type = 2;
  string ip = 3;
  int32 udp_port = 4;
  bool controllable = 5;
  string status = 6;
}
```

Comando (Gateway→Fonte, TCP)

```
message Comando {
  string source_id = 1;
  string acao = 2; // "ativar"
  float valor = 3; // set_freq
}
```

RespostaComando (Fonte→Gateway)

```
message RespostaComando {
  bool sucesso = 1;
  string mensagem = 2;
}
```

Leitura (Fonte→Gateway, UDP)

```
message Leitura {
  string source_id = 1;
  string type = 2;
  int64 timestamp = 3;
  oneof valor { // so um
    float temperatura = 4;
    float umidade = 5;
    float co2 = 6;
    float ruido = 7;
    int32 contagem = 8;
    float energia = 9;
  }
  bool alerta = 10;
}
```

RequisicaoCliente (Java→Gateway, TCP)

```
message RequisicaoCliente {
  string tipo = 1; // "listar"
  Comando comando = 2;
  string consulta = 3; // "media_temp_1h"
}
```

RespostaGateway (Gateway→Java, TCP)

```
message RespostaGateway {
  string tipo = 1;
  repeated DiscoveryResponse fontes = 2;
  RespostaComando resultado_cmd = 3;
  string dados_consulta = 4;
}
```

Framing TCP e Fluxo de Descoberta

Protocolo de framing — identico em Python, C e Java

Todo envio TCP e prefixado com **4 bytes big-endian** indicando o tamanho do payload protobuf. Sem framing, o TCP poderia fundir ou fragmentar mensagens.

Offset	0	1	2	3	4 ... N+3
Conteudo	N >> 24	N >> 16	N >> 8	N & 0xFF	Payload protobuf serializado (N bytes)

Linguagem	Envio (4 bytes + payload)	Recebimento
Python	struct.pack('>I', len(data)) + data	struct.unpack('>I', recv(4))[0] → recv(N)
C	buf[0..3] = htonl(sz); send(buf,4); send(proto,sz)	recv(4) → ntohl → recv(N)
Java	(int >> 24,16,8,0) escrito manual em DataOutputStream	DataStream.readInt() → readFully(N)

Sequencia de descoberta

- 1. Boot:** Gateway envia `DiscoveryRequest{ip, port=6000}` via UDP multicast 224.1.1.1:5007
- 2. Fontes continuas** (sensor_temp, qualidade_ar): respondem com `DiscoveryResponse` via UDP unicast → 6002
- 3. Fontes controlaveis** (camera, semaforo, poste): conectam TCP → 6000 e enviam `DiscoveryResponse`
- 4. Gateway** persiste cada fonte no SQLite e inicia coleta de Leituras periodicas

Gateway Inteligente — Python 3

Processo unico com **4 threads** concorrentes; estado compartilhado protegido por **threading.Lock**. Ponto central de todas as comunicacoes.

Thread	Porta	Funcao principal
<code>t_udp</code>	:6002 UDP	Recebe DiscoveryResponse (fontes continuas) + Leitura; persiste no SQLite
<code>t_tcp_fontes</code>	:6000 TCP	Aceita conexoes de fontes controlaveis; encaminha Comando e recebe RespostaComando
<code>t_tcp_cliente</code>	:6001 TCP	Atende 1 cliente Java; processa listar / comando / consulta
<code>t_api</code>	:6003 HTTP	REST API (http.server); serve dados para o Dashboard Web

Bibliotecas Python

Biblioteca	Uso
<code>protobuf 5.x</code>	Serializar/deserializar todas as mensagens
<code>sqlite3</code>	Persistencia (stdlib)
<code>socket</code>	UDP + TCP (stdlib)
<code>threading</code>	4 threads concorrentes (stdlib)
<code>struct</code>	Framing: pack/unpack 4 bytes (stdlib)
<code>http.server</code>	API REST sem framework externo

Trecho de codigo: envio de comando TCP

```
cmd = cidade_pb2.Comando()
cmd.source_id = source_id
cmd.acao = "ativar"
data = cmd.SerializeToString()
size = struct.pack('>I', len(data))
sock.sendall(size + data)
# leitura da resposta:
raw = recv_all(sock, 4)
n = struct.unpack('>I', raw)[0]
buf = recv_all(sock, n)
resp = cidade_pb2.RespostaComando()
resp.ParseFromString(buf)
```

Funcao `recv_all(sock, n)` garante leitura completa de N bytes, necessaria pois `recv()` pode retornar menos que o solicitado.

Fontes de Dados — C (GCC 15 + protobuf-c 1.5)

5 processos independentes compilados via **Makefile** com `-lprotobuf-c`. Cada processo e uma instancia separavel e executavel em qualquer host da rede.

Processo	Tipo	Dado (campo oneof)	Comandos aceitos	Periodicidade
sensor_temperatura	Nao ctrl.	temperatura (float)	—	15 s
sensor_qualidade_ar	Nao ctrl.	co2 (float)	—	10 s
camera	Controlavel	contagem (int32)	ativar, desativar, set_freq	5 s
semaforo	Controlavel	contagem (int32)	ativar, desativar, set_freq	3 s
poste	Controlavel	energia (float)	ativar, desativar, set_limiar	8 s

Arquitetura interna (fontes controlaveis)

- Thread principal: loop UDP — envia Leitura a cada N segundos
- Thread TCP: escuta comandos do Gateway em loop bloqueante
- Variavel global ativo + mutex sincroniza as threads
- Variaveis frequencia e limiar ajustadas em tempo real pelo comando

Bypass WSL2 (arg)

Como multicast nao cruza a fronteira WSL2↔Windows, todos os processos aceitam argumentos `<ip> <porta>` para conexao direta, ignorando a fase de descoberta:

```
int main(int argc, char *argv[]) {
    if (argc >= 3) {
        // conecta TCP direto
        connect(tcp_fd, &gw, sizeof(gw));
        // registra sem multicast
        enviar_discovery_response();
    } else {
        descobrir_e_registrar(); // multicast
    }
}
```

Trecho: envio de Leitura em C

```
Cidade__Leitura msg =
    CIDADE__LEITURA__INIT;
msg.source_id = SENSOR_ID;
msg.type = "temperatura";
msg.timestamp = time(NULL);
msg.temperatura = gerar_temp();
msg.valor_case =
    CIDADE__LEITURA__VALOR_TEMPERATURA;
msg.alerta = (msg.temperatura > LIMIAR);

size_t sz = cidade__leitura__
    get_packed_size(&msg);
uint8_t *buf = malloc(sz);
cidade__leitura__pack(&msg, buf);

// framing: 4 bytes big-endian
uint32_t nl = htonl((uint32_t)sz);
send(tcp_fd, &nl, 4, 0);
send(tcp_fd, buf, sz, 0);
```

Cliente Analitico — Java 21

Processo Java independente, executado com `java -jar cliente-1.0.jar`. Conecta ao Gateway via TCP na porta 6001 usando framing identico ao Python e C.

Op.	Descricao	Mensagem enviada (RequisicaoCliente)
1	Listar fontes conectadas	<code>tipo="listar"</code>
2	Ativar uma fonte	<code>tipo="comando", cmd.acao="ativar"</code>
3	Desativar uma fonte	<code>tipo="comando", cmd.acao="desativar"</code>
4	Alterar frequencia de envio	<code>tipo="comando", cmd.acao="set_frequencia", cmd.valor=N</code>
5	Alterar limiar de alerta	<code>tipo="comando", cmd.acao="set_limiar", cmd.valor=N</code>
6	Media temperatura ultima 1h	<code>tipo="consulta", consulta="media_temp_1h"</code>
7	Desvio padrao CO2 ultimas 24h	<code>tipo="consulta", consulta="desvio_co2_24h"</code>

Stack Java

Componente	Versao / Detalhe
Java	21 (LTS)
Maven	3.9 (build + dep)
protobuf-java	4.34.1 (com.google.protobuf)
maven-shade-plugin	uber-JAR com dependencias
protoc	4.34.1 — gera Cidade.java

Trecho: envio e leitura de resposta

```
// Envio com framing
RequisicaoCliente req =
RequisicaoCliente.newBuilder()
.setTipo("listar")
.build();
byte[] payload = req.toByteArray();
out.writeInt(payload.length);
out.write(payload);
out.flush();
```

```
// Leitura da resposta
int n = in.readInt();
byte[] buf = new byte[n];
in.readFully(buf);
RespostaGateway resp =
RespostaGateway.parseFrom(buf);
```

Persistencia SQLite + API REST

Banco de Dados — cidade.db

Tabela leituras

```
CREATE TABLE leituras (  
  id INTEGER PRIMARY KEY,  
  source_id TEXT,  
  type TEXT,  
  valor REAL,  
  alerta INTEGER,  
  timestamp INTEGER  
);
```

Tabela fontes

```
CREATE TABLE fontes (  
  source_id TEXT PRIMARY KEY,  
  type TEXT,  
  ip TEXT,  
  status TEXT,  
  controllable INTEGER,  
  ultima_leitura INTEGER  
);
```

Consultas analíticas implementadas

- Média por tipo e intervalo de tempo
- Desvio padrão por tipo e intervalo
- Fonte com maior variação de valores
- Total de alertas por fonte

API REST — Porta 6003

Metodo	Rota	Descrição
GET	/fontes	Lista todas as fontes
GET	/historico?type=&segundos=	Leituras recentes filtradas
GET	/alertas	Últimos alertas
GET	/consultas/media?type=	Média por tipo
GET	/consultas/desvio?type=	Desvio padrão
GET	/consultas/major_variacao	Fonte mais variável
GET	/consultas/alertas	Total alertas/fonte
POST	/comando	Envia comando a fonte

Dashboard Web

- Arquivo único dashboard.html (sem build)
- Chart.js 4.4 (CDN) — gráficos de séries temporais
- Polling a cada 5 s na API REST
- Envia comandos via POST /comando
- CORS: Access-Control-Allow-Origin: *

Stack completo de tecnologias

Camada	Tecnologia	Versao	Papel no sistema
Serializacao	Protocol Buffers	4.34.1	Todas as msgs entre todos os processos
Gateway	Python	3.14	Orquestrador central; threading + sqlite3
Fontes	C / GCC	15.2	Processos nativos de alta performance
Fontes lib	protobuf-c	1.5.1	Serializacao protobuf em C puro
Cliente	Java	21 LTS	JVM; DataInputStream/OutputStream
Build Java	Maven + shade-plugin	3.9	Uber-JAR com todas as deps embutidas
Persistencia	SQLite	stdlib	Banco embutido, zero configuracao
API	http.server	stdlib	Servidor HTTP sem framework externo
Dashboard	Chart.js	4.4 (CDN)	Graficos no browser; polling REST